

PRD-05

Placement Ecosystem + Resume Builder + Events + Notifications + Analytics + Admin Platform

Version 1.0

Objective

Develop the final core backend modules that transform the CSE Student Platform into a production-ready ecosystem. This phase introduces placement preparation, resume management, event scheduling, notifications, analytics, and a comprehensive administration panel while leveraging the event-driven architecture introduced in PRD-04.

Scope

This PRD includes:

- Placement Hub
 - Company Management
 - Job & Internship Listings
 - Application Tracker
 - Resume Builder
 - Resume Templates
 - Event Management
 - Notifications
 - User Analytics
 - Admin Dashboard
 - Platform Reports
 - System Settings
-

Database Models

Company

Note: Reuse the [Company](#) model created in PRD-03. Extend it instead of creating a duplicate.

Add:

- description
 - industry
 - headquarters
 - careersUrl
 - logo
 - verified
-

JobPosting

Fields:

- id
 - companyId
 - title
 - type (Internship / Full-Time / Part-Time)
 - location
 - workMode (Remote / Hybrid / Onsite)
 - description
 - requirements
 - salaryRange
 - applicationUrl
 - applicationDeadline
 - experienceRequired
 - isPublished
 - createdAt
-

JobApplication

Fields:

- id
- userId
- jobId
- status
- appliedAt

- notes

Status:

- Saved
 - Applied
 - OA Scheduled
 - Interview
 - HR Round
 - Offered
 - Rejected
 - Withdrawn
-

Resume

Fields:

- id
 - userId
 - title
 - template
 - resumeUrl
 - atsScore
 - isDefault
 - updatedAt
-

ResumeSection

Fields:

- id
- resumeld
- sectionType
- content (JSON)

Examples:

- Education
 - Experience
 - Skills
 - Projects
 - Certifications
 - Achievements
-

Event

Fields:

- id
- title
- description
- type
- organizer
- location
- startTime
- endTime
- registrationUrl
- maxParticipants
- isPublished

Types:

- Hackathon
 - Webinar
 - Workshop
 - Contest
 - Bootcamp
 - Meetup
-

EventRegistration

Fields:

- id
 - eventId
 - userId
 - registeredAt
-

Notification

Fields:

- id
- userId
- title
- message
- type
- isRead
- metadata (JSON)
- createdAt

Types:

- Placement
 - Project
 - Coding
 - Learning
 - Event
 - System
-

UserAnalytics

Instead of storing derived metrics permanently, persist only metrics that cannot be efficiently recalculated.

Suggested fields:

- id
- userId
- lastLogin
- totalStudyMinutes
- currentLearningStreak
- longestLearningStreak

Continue deriving solved counts, project counts, and application counts from existing tables.

PlatformSetting

Key-value configuration table.

Examples:

- maintenanceMode
 - registrationEnabled
 - maxTeamMembers
 - defaultXP
 - featureFlags
-

APIs

Companies

GET /api/companies

GET /api/companies/:id

POST /api/companies
PUT /api/companies/:id
DELETE /api/companies/:id

Jobs

GET /api/jobs
GET /api/jobs/:id
POST /api/jobs
PUT /api/jobs/:id
DELETE /api/jobs/:id

Filters:

- Company
- Location
- Type
- Work Mode
- Experience
- Search

Pagination required.

Job Applications

POST /api/job-applications
GET /api/job-applications
PATCH /api/job-applications/:id
DELETE /api/job-applications/:id

Resume

GET /api/resumes
GET /api/resumes/:id
POST /api/resumes
PUT /api/resumes/:id
DELETE /api/resumes/:id
POST /api/resumes/:id/default

Store resume files in **Supabase Storage**.

Resume Sections

POST /api/resume-sections
PUT /api/resume-sections/:id

DELETE /api/resume-sections/:id

Events

GET /api/events
GET /api/events/:id
POST /api/events
PUT /api/events/:id
DELETE /api/events/:id

Event Registration

POST /api/events/:id/register
DELETE /api/events/:id/register
GET /api/my-events

Notifications

GET /api/notifications
PATCH /api/notifications/:id/read
PATCH /api/notifications/read-all
DELETE /api/notifications/:id

Notifications should be created through the event system introduced in PRD-04 rather than directly from business services.

Analytics

GET /api/analytics/dashboard

Return a unified dashboard including:

- Learning progress
- Coding statistics
- Active projects
- Placement progress
- Resume completion
- Event registrations
- Current streaks

Derive metrics from existing modules whenever possible.

Admin Dashboard

GET /api/admin/dashboard

Include:

- Total Users
 - Active Users
 - Published Roadmaps
 - Coding Problems
 - Projects
 - Teams
 - Jobs
 - Events
 - Platform Growth
 - Daily Registrations
-

Reports

GET /api/admin/reports

Support:

- Date range
 - CSV export (future extension)
 - Summary statistics
-

Platform Settings

GET /api/admin/settings

PUT /api/admin/settings

Business Rules

- One default resume per user.
 - Duplicate event registrations are not allowed.
 - Event capacity must not exceed `maxParticipants`.
 - Applications are unique per user/job pair.
 - Notifications are generated from domain events.
 - Resume files are stored in Supabase Storage.
 - Analytics derive data from existing modules where practical.
 - Only admins can manage companies, jobs, events, reports, and platform settings.
-

Validation

Use Zod for:

- URLs
 - Dates
 - Salary range
 - Enum values
 - Resume metadata
 - Event capacity
 - Platform setting values
-

Authorization

Student

- Browse companies and jobs
- Track applications
- Manage resumes
- Register for events
- Read notifications
- View analytics

Mentor

- Student permissions
- View learner analytics (future extension)

Admin

- Full CRUD across companies, jobs, events, reports, and settings
 - View platform-wide analytics
-

Performance Requirements

- Pagination for all list endpoints.
- Add indexes on:
 - companyId
 - jobId
 - userId
 - eventId

- applicationDeadline
 - startTime
 - notification type
 - Continue avoiding N+1 queries.
 - Keep analytics queries optimized through aggregation.
-

API Response Standard

Continue using the response structure defined in PRD-01.

Acceptance Criteria

PRD-05 is complete when:

- All new Prisma models and migrations are successfully applied.
 - CRUD APIs for jobs, resumes, events, notifications, and settings are implemented.
 - Company model is extended without duplication.
 - Job applications, resume management, and event registration workflows function correctly.
 - Notifications are generated through the existing event-driven architecture.
 - Analytics aggregate data from all previous modules.
 - Admin dashboard and reports are operational.
 - Validation, authorization, logging, and Swagger documentation are updated.
 - The backend compiles without TypeScript errors and integrates cleanly with PRD-01 through PRD-04.
-

After PRD-05

At this point, I would **stop adding major backend features** and shift focus to production readiness instead of expanding functionality.

The next phase should not be "PRD-06 Features." It should be an **Engineering Hardening Phase**, including:

- Redis caching
- Background job queues (BullMQ)
- Email queue
- WebSocket gateway for live notifications
- Object caching and rate limiting optimization
- Monitoring (Prometheus/Grafana or similar)
- Audit logging
- CI/CD pipelines
- Automated testing (unit, integration, end-to-end)
- Docker and Docker Compose

- Production environment configuration
- Database backup and recovery strategy
- API versioning
- Security review (OWASP)
- Performance benchmarking
- Load testing
- CDN integration
- Horizontal scaling readiness

That sequence mirrors how mature software teams transition from feature development to preparing a system for real-world usage and growth.